PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Instituto de Ciências Exatas e Informática

Disciplina	Curso	Período
Algoritmos e Estruturas de Dados II	Engenharia de Computação	2°

Professor
Kleber Jacques F. de Souza (klebersouza@pucminas.br)

Lista de Exercícios 1 - Respostas

Instruções

- Esta é uma lista de reforço.
- Cada sessão tem por objetivo treinar uma característica dos algoritmos.
- Tente fazer sozinho.

1 Arquivos

1. Faça um programa que crie um arquivo TEXTO em disco, com o nome “dados.txt", e escreva neste arquivo em disco uma contagem que vá de 1 até 100, com um número em cada linha. Abra este arquivo em um editor de textos, como por exemplo o Notepad ou o Wordpad do Windows. Faça outro programa que seja capaz de abrir o arquivo “dados.txt"e exiba seu conteúdo na tela.

   Resposta:

```c
#include <stdio.h>
#include <stdlib.h>
int main(){

    FILE *arq_grava = fopen("dados.txt","w");

    if(arq_grava == NULL){
        printf("Erro ao gerar o arquivo!");
        exit(0);
    }

    for(int i=1; i<=100; i++)
        fprintf(arq_grava,"%d\n",i);

    fclose(arq_grava);

    FILE *arq_ler = fopen("dados.txt","r");

    if(arq_ler == NULL){
        printf("Erro ao abrir o arquivo!");
        exit(0);
    }

    char dados[101];
    for(int i=1; i<=100; i++){
        fgets(dados,100,arq_ler);
            printf("%s",dados);
    }

```

```
    fclose(arq_ler);

    return 0;
}
```

2. Faça um programa que crie um arquivo BINÁRIO em disco, com o nome “dados.dat", e escreva neste arquivo em disco uma contagem que vá de 1 até 100, com um número em cada linha. Abra este arquivo em um editor de textos, como por exemplo o Notepad ou o Wordpad do Windows. Faça outro programa que seja capaz de abrir o arquivo “dados.dat"e exiba seu conteúdo na tela.

   Resposta:

```
#include <stdio.h>
#include <stdlib.h>
int main(){

    FILE *arq_grava = fopen("dados.dat","wb");

    if(arq_grava == NULL){
        printf("Erro ao gerar o arquivo!");
        exit(0);
    }

    char quebra_linha = '\n';
    for(int i=1; i<=100; i++){
        fwrite(&i, sizeof(int),1, arq_grava);
        fwrite(&quebra_linha, sizeof(char),1, arq_grava);
    }

    fclose(arq_grava);

    FILE *arq_ler = fopen("dados.dat","rb");

    if(arq_ler == NULL){
        printf("Erro ao abrir o arquivo!");
        exit(0);
    }

    int dados;
    for(int i=1; i<=100; i++){
        fread(&dados, sizeof(int),1, arq_grava);
        fread(&quebra_linha, sizeof(char),1, arq_grava);
        printf("%d\n",dados);
    }

    fclose(arq_ler);

    return 0;
}
```

3. Faça um programa que imite um editor de textos. Inicialmente você irá ler os dados digitados pelo usuário (linhas de texto) e criar um vetor em memória onde serão armazenados os textos fornecidos pelo usuário (texto de 1 até no máximo 50 linhas). O usuário vai escrever o seu texto, terminando por uma linha onde ele irá escrever apenas a palavra ‘FIM’, o que determina que ele não deseja mais digitar linhas de texto. Sendo assim, o texto final pode ter um número variável de linhas, entre 1 e 50. Salvar o conteúdo armazenado na memória neste vetor, em um arquivo texto em disco. Sugestão: use o comando “gets"para ler as linhas de texto digitadas pelo usuário.

   Resposta:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(){

    char texto[50][100];
    int indice=0;

    printf("\n*************** Editor de Texto ***************\n");
    printf("Digite o TEXTO desejado,\nmaximo 50 linhas (100 caracteres por
       linha)\n");
    printf("Para finalizar escreva FIM em uma linha\n\n");

    gets(texto[indice++]);

    while(indice < 50 && strcmp(texto[indice-1],"FIM")!=0)
        gets(texto[indice++]);

    FILE *arquivo = fopen("dados.txt","w");

    if(arquivo == NULL){
        printf("Erro ao abrir o arquivo!");
        exit(0);
    }

    for(int i=0; i<indice; i++)
         fprintf(arquivo,"%s\n",texto[i]);

    fclose(arquivo);

    printf("Arquivo salvo com sucesso!\n");

    return 0;
}
```

4. Faça um programa que leia (do teclado) um cadastro de 10 alunos, indicando o nome, nota1, nota2. Calcule a média aritmética simples dos 10 alunos e depois escreva em um arquivo os dados de cada aluno: nome, nota1, nota2 e média. Lembre-se de que as notas e média deverão ser apresentadas como valores que possuem até 2 casas após a vírgula.

   Resposta:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct ALUNO{
    char nome[100];
    float nota1;
    float nota2;
    float media;
};
const int MAX_ALUNO = 2;
int main(){

    struct ALUNO alunos[MAX_ALUNO];
    char aux[100];

    for(int i=0; i<MAX_ALUNO; i++){

        printf("\nDigite o nome do %d Aluno: ",i+1);
```

```
        gets(alunos[i].nome);

        printf("Digite a nota1 do %d Aluno: ",i+1);
        gets(aux);
        alunos[i].nota1 = atof(aux);

        printf("Digite a nota2 do %d Aluno: ",i+1);
        gets(aux);
        alunos[i].nota2 = atof(aux);

        alunos[i].media = (alunos[i].nota1+alunos[i].nota2)/2;
    }

    FILE *arquivo = fopen("alunos.dat","wb");
    fwrite(alunos, sizeof(struct ALUNO),MAX_ALUNO,arquivo);
    fclose(arquivo);

    // ler dados

    struct ALUNO alunos_ler[MAX_ALUNO];
    FILE *arquivo_ler = fopen("alunos.dat","rb");
    fread(alunos_ler, sizeof(struct ALUNO),MAX_ALUNO,arquivo_ler);
    fclose(arquivo_ler);

    for(int i=0; i<MAX_ALUNO; i++){

        printf("\nAluno: %d\n",i+1);

        printf("Nome: %s\n",alunos_ler[i].nome);

        printf("nota1: %.2f\n",alunos_ler[i].nota1);

        printf("nota2: %.2f\n",alunos_ler[i].nota2);

        printf("Media: %.2f\n\n",alunos_ler[i].media);
    }

    return 0;
}
```

5. Faça um programa de criptografia de dados, ou seja, um programa capaz de ler um arquivo texto, codificar este arquivo através de alguma técnica de alteração do código ASCII (exemplo: somar 1 ao valor ASCII de cada carácter), e escrever em disco o arquivo codificado. Crie um outro programa que descriptografe um arquivo criado pelo programa de criptografia, realizando a operação inversa: ler o arquivo do disco, descodificar e escrever o novo arquivo em disco descriptografado. Lembre-se que para que seja possível criptografar/descriptografar um arquivo a função de codificação deve possuir uma função inversa. Exemplo de código de criptografia:

```
tamanho = strlen(linha);
for (i=0; i < tamanho; i++){
        // Soma 1 ao código de cada caracter da string
        linha_cripto[i] = linha[i] + 1;
}
// Adiciona a marca de fim de string
linha_cripto[tamanho]='\0';
```

Desafios: Você consegue desenvolver uma função de criptografia/descriptografia que seja mais sofisticada e menos óbvia do que esta? Você conseguiria adaptar este programa para criptografar qualquer tipo de arquivo, seja ele um arquivo texto ou binário?

Resposta:

```c
#include <stdio.h>
#include <stdlib.h>
int main(){

    FILE * arquivo = fopen("dados.txt","rb");
    FILE * arq_cripto = fopen("dados_cript.txt","wb");

    if(!arquivo || !arq_cripto){
        printf("Erro ao abrir o arquivo\n");
        exit(0);
    }

    int c;

    while (( c = fgetc( arquivo )) != EOF )
        putc (c+1,arq_cripto);

    fclose(arquivo);
    fclose(arq_cripto);


    FILE *arq_cripto_ler = fopen("dados_cript.txt","rb");
    FILE *arq_descripto = fopen("dados_descript.txt","wb");

    while (( c = fgetc( arq_cripto_ler )) != EOF )
        putc (c-1,arq_descripto);

    fclose(arq_cripto_ler);
    fclose(arq_descripto);

    return 0;
}
```

2 Ponteiros e Alocação Dinâmica de Memória

1. Faça um programa que leia um valor n e crie dinamicamente um vetor de n elementos e passe esse vetor para uma função que vai ler os elementos desse vetor. Depois, no programa principal, o vetor preenchido deve ser impresso. Além disso, antes de finalizar o programa, deve-se liberar a área de memória alocada.

   Resposta:

```c
#include <stdio.h>
#include <stdlib.h>

int *aloca_vetor(int n){

    int *vetor = (int *)calloc(n,sizeof(int));

    if(vetor == NULL){
        printf("Espaco de Memoria Insuficiente!");
        exit(0);
    }

    return vetor;
}
```

```c

void preenche_vetor(int *v, int n){
    for(int i=0;i<n;i++)
        v[i] = rand()%n+1;
}

void imprimi_vetor(int *v, int n){
    for(int i=0;i<n;i++)
        printf("%d\n",v[i]);
}

int main(){

    int n = 10;
    int *novo_vetor = aloca_vetor(n);
    preenche_vetor(novo_vetor, n);
    imprimi_vetor(novo_vetor, n);
    free(novo_vetor);

    return 0;
}
```

2. Faça uma função que receba um valor n e crie dinamicamente um vetor de n elementos e retorne um ponteiro. Crie uma função que receba um ponteiro para um vetor e um valor n e imprima os n elementos desse vetor. Construa também uma função que receba um ponteiro para um vetor e libere esta área de memória. Ao final, crie uma função principal que leia um valor n e chame a função criada acima. Depois, a função principal deve ler os n elementos desse vetor. Então, a função principal deve chamar a função de impressão dos n elementos do vetor criado e, finalmente, liberar a memória alocada através da função criada para liberação.

   Resposta:

```c
#include <stdio.h>
#include <stdlib.h>

int *aloca_vetor(int n){

    int *vetor = (int *)calloc(n,sizeof(int));

    if(vetor == NULL){
        printf("Espaco de Memoria Insuficiente!");
        exit(0);
    }

    return vetor;
}

void libera_vetor(int *v){
    free(v);
}

void preenche_vetor(int *v, int n){
    for(int i=0;i<n;i++)
        v[i] = rand()%n+1;
}

void imprimi_vetor(int *v, int n){
    for(int i=0;i<n;i++)
        printf("%d\n",v[i]);
```

```
}

int main(){

    int n = 10;
    int *novo_vetor = aloca_vetor(n);
    preenche_vetor(novo_vetor, n);
    imprimi_vetor(novo_vetor, n);
    libera_vetor(novo_vetor);

    return 0;
}
```

3. Construa um programa que aloque em tempo de execução (dinamicamente) uma matriz de ordem m x n (linha por coluna), usando $1+m$ chamadas a função malloc. Agora, aproveite este programa para construir uma função que recebendo os parâmetros m e n aloque uma matriz de ordem m x n e retorne um ponteiro para esta matriz alocada. Crie ainda uma função para liberar a área de memória alocada pela matriz. Finalmente, crie um novo programa que teste/use as duas funções criadas acima.

   Resposta:

```
#include <stdio.h>
#include <stdlib.h>

int **aloca_matriz(int m, int n){

    int **matriz = (int**)malloc(m*sizeof(int *));

    for(int i=0;i<m;i++)
        matriz[i] = (int*)malloc(n*sizeof(int));

    return matriz;
}

void desaloca_matriz(int **matriz, int m){

    for(int i=0;i<m;i++)
        free(matriz[i]);

    free(matriz);
}

void preenche_matriz(int **matriz, int m, int n){

    for(int i=0;i<m;i++)
        for(int j=0;j<n;j++)
        matriz[i][j] = i+j;

}

void imprimi_matriz(int **matriz, int m, int n){

    for(int i=0;i<m;i++){
        for(int j=0;j<n;j++)
        printf("%d\t",matriz[i][j]);

        printf("\n");
    }
}

```

```c
int main(){

    int m = 3;
    int n = 5;

    int **nova_matriz = aloca_matriz(m,n);
    preenche_matriz(nova_matriz,m,n);
    imprimi_matriz(nova_matriz,m,n);
    desaloca_matriz(nova_matriz,m);

    return 0;
}
```

4. Criar um tipo abstrato de dados (ESTRUTURA) que represente uma pessoa, contendo nome, data de nascimento e CPF. Crie uma variável que é um ponteiro para este TAD (no programa principal). Depois crie uma função que receba este ponteiro e preencha os dados da estrutura e também uma uma função que receba este ponteiro e imprima os dados da estrutura. Finalmente, faça a chamada a esta função na função principal.

   Resposta:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct PESSOA{
    char nome[100];
    char data_nasc[15];
    char cpf[15];
};

struct PESSOA * aloca_pessoa(){

    struct PESSOA *new_p = (struct PESSOA *)malloc(sizeof(struct PESSOA));

    if(new_p == NULL){
        printf("Memoria Insufuciente!");
        exit(0);
    }

    return new_p;
}

void desaloca_pessoa(struct PESSOA *p){
    free(p);
}

void preenche_pessoa(struct PESSOA *p){

    strcpy(p->nome, "Pedro de Alcantara");
    strcpy(p->data_nasc, "12/10/1798");
    strcpy(p->cpf, "000.000.000-00");
}

void imprimi_pessoa(struct PESSOA *p){

    printf("Nome: %s\n", p->nome);
    printf("Data de Nascimento: %s\n", p->data_nasc);
    printf("CPF: %s\n", p->cpf);
}

```

```c
int main(){

    struct PESSOA *p;

    p = aloca_pessoa();
    preenche_pessoa(p);
    imprimi_pessoa(p);
    desaloca_pessoa(p);

    return 0;
}
```

5. O que significa o operador asterisco '*' em cada um dos seguintes casos:

   (a) int *p;
   Resposta: Declaração de uma variável do **tipo ponteiro para inteiro**.

   (b) printf("%d", *p);
   Resposta: Imprimi o **conteúdo apontado pelo ponteiro p**, no formato de um dígito.

   (c) *p = x*5;
   Resposta: O **conteúdo do ponteiro p** recebe o valor da variável x multiplicado por 5.

   (d) printf("%d", *(p+1));
   Resposta: Imprime o **conteúdo apontado pela próxima posição do ponteiro p** no formato de número.

3 Análise de Complexidade

1. Suponha dois algoritmos A e B, com funções de complexidade:

   (a) $A(n) = n^2 - n + 289$

   Resposta:

$$O(n^2) - O(n) + O(289) = O(n^2)$$

   (b) $B(n) = 7n + 49$
   Resposta:

$$O(7n) + O(49) = O(n)$$

   Determine, utilizando a notação O o grau de complexidade de cada um dos algoritmos.

2. Determine a complexidade o programa abaixo. O que esse algoritmo faz?

```c
int main (){
        int i=0, n=0, s=0, V[100];
        scanf(" %d",&n);
        do{
                scanf("%d",V[i]);
                s = s + V[i++];
        }while(i<n);
        cout<<s/n;
}
```

   Resposta:

   O algoritmo calcula a média dos valores recebidos no vetor V.

$$O(1) + O(1) + O(n) + O(n) * O(1) + O(n) * O(1) + O(1) = O(n);$$

3. O que significa dizer: $O(f(n)) + O(g(n)) = O(max(f(n), g(n)))$?

   Resposta: Esta é um propriedade da notação O que significa que a soma de duas funções da notação O $(f(n) e g(n))$ será representada pelo valor máximo entre as duas funções. Ou seja, a soma representa o custo máximo para executar as duas funções separadamente.

4. Determine a (análise de) complexidade de cada um dos algoritmos abaixo e indique sua ordem de complexidade. Considere o melhor e o pior caso para cada um.

   (a) Somar matrizes

```
for(i=0; i<n; i++)
  for(j=0; j<n; j++)
    a[i][j] = b[i][j] + c[i][j];
```

   Resposta:
   Melhor Caso: $O(n^2)$
   Pior Caso: $O(n^2)$

   (b) Multiplicar matrizes

```
for(i=0; i<n; i++)
  for(j=0; j<n; j++)
  {
    a[i][j] = 0;
    for(k=0; k<n; k++)
      a[i][j] += b[i][k] * c[k][j];
  }
```

   Resposta:
   Melhor Caso: $O(n^3)$
   Pior Caso: $O(n^3)$

   (c) Encontrar o maior valor de um vetor

```
maior = vet[0];
for(i=1; i<n; i++)
  if(vet[i]>valor)
    maior = vet[i];
```

   Resposta:
   Melhor Caso: $O(n)$
   Pior Caso: $O(n)$

   (d) Encontrar o maior e o menor valor de um vetor

```
maior = vet[0];
menor = vet[0];
for(i=1; i<n; i++)
  if(vet[i]>valor)
    maior = vet[i];
  else
    if(vet[i]<menor)
      menor = vet[i];
```

   Resposta:
   Melhor Caso: $O(n)$
   Pior Caso: $O(n)$

   (e) Transpor matrizes

```
for(i=0; i<n; i++)
  for(j=i+1; j<n; j++)
  {
    tmp = a[i][j];
    a[i][j] = a[j][i];
    a[j][i] = tmp;
  }
```

Resposta:
Melhor Caso: $O(n^2)$
Pior Caso: $O(n^2)$

(f) Laço de repetição maluco

```
cont = 0;
for(i=1; i<=n; i*=2)
  for(j=0; j<n; j++)
    cont++;
```

Resposta:
Melhor Caso: $O(n \log n)$ Pior Caso: $O(n \log n)$

(g) Encontrar o k-ésimo valor em um vetor não ordenado

```
int selec_kesimo (int a[], int k, int n)
{
  int i, j, mini, tmp;
  for(i=0; i<k; i++)
  {
    mini = i;
    for(j=i+1; j<n; j++)
      if(a[j]<a[mini])
        mini = j;
    tmp = a[i];
    a[i] = a[mini];
    a[mini] = tmp;
  }
  return a[k-1];
}
```

Resposta:
Melhor Caso: $O(n^2)$
Pior Caso: $O(n^2)$

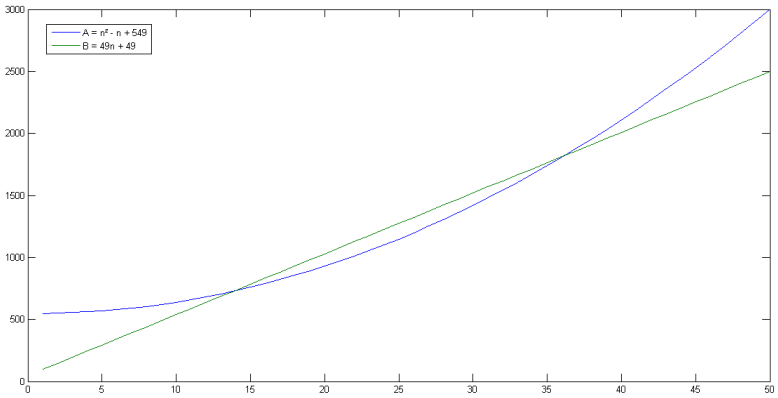
5. Suponha um algoritmo A e um algoritmo B com funções de complexidade de tempo $a(n) = n^2 - n + 549$ e $b(n) = 49n + 49$, respectivamente. Determine quais são os valores de n pertencentes ao conjunto dos números naturais para os quais A leva menos tempo para executar do que B. É correto afirmar que B domina assintoticamente A para quaisquer valores de n? Justifique.

   Resposta: Os valores de n nos quais A leva menos tempo para executar que B são: $14 \leq n \leq 36$. Não é correto afirmar que B domina assintoticamente A para quaisquer valor de n, pois existe valores para n, onde A leva menos tempo que B e outros em que B leva menos tempo que A. É correto afirma, por exemplo, que A domina assintoticamente B para $n \geq 37$.

4 Recursividade

1. O máximo divisor comum (MDC) de dois números inteiros x e y pode ser calculado usando-se uma definição recursiva:

$$MDC(x, y) = MDC(x - y, y), \ se \ x > y$$



Além disso, sabe-se que:

$$MDC(x, y) = MDC(y, x)$$

$$MDC(x, x) = x$$

Exemplo: $MDC(10, 6) = MDC(4, 6) = MDC(6, 4) = MDC(2, 4) = MDC(4, 2) = MDC(2, 2) = 2$

Crie uma função recursiva para descrever tal definição. Crie, também, um algoritmo que leia os dois valores inteiros e utilize a função criada para calcular o MDC de x e y, e imprima o valor computado.

Resposta:

```
int MDC(int x, int y){

    if(x == y)
        return x;
    else
        if(x > y)
            return MDC(x - y, y);
        else return MDC(y - x, x);
}
```

2. Considere um sistema numérico que não tenha a operação de adição implementada e que você disponha somente dos operadores (funções) sucessor e predecessor. Então, pede-se para escrever uma função recursiva que calcule a soma de dois números x e y através desses dois operadores: sucessor e predecessor.

   Resposta:

```
int soma(int x, int y){

    if(x==0)
        return y;
    if(y==0)
        return x;

    if(x > y)
        soma(sucessor(x),predecessor(y));
    else
        soma(predecessor(x),sucessor(y));
}
```

3. Implemente uma função recursiva que, dados dois números inteiros x e n, calcula o valor de x^n.

   Resposta:

```c
int potencia(int x, int n){

    if(n==0)
        return 1;
    else return x * potencia(x,n-1);

}
```

4. Considere uma partida de futebol entre duas equipes A x B, cujo placar final é m x n, em que m e n são os números de gols marcados por A e B, respectivamente. Implemente um algoritmo recursivo que imprima todas as possíveis sucessões de gols marcados. Por exemplo, para um resultado de 3 x 1 as possíveis sucessões de gols são “A A A B", “A A B A", “A B A A"e “B A A A".

   Resposta:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void gols_marcados(char *gols, int i, int m, int n){

    if(m >= 1){
        gols[i]='A';
        gols_marcados(gols,i+1,m-1,n);
    }

    if(n >= 1){
        gols[i]='B';
        gols_marcados(gols,i+1,m,n-1);
    }

    if(m==0 && n==0){
        gols[i]=0;
        printf("%s\n",gols);
    }
}

int main(){
    int m=3;
    int n=1;
    char gols[m+n+1];
    gols_marcados(gols,0,m,n);
    return 0;
}
```

5. Dado um vetor de inteiros A, apresente algoritmos recursivos para calcular:

   (a) o elemento máximo do vetor
   (b) o elemento mínimo do vetor
   (c) a soma dos elementos do vetor
   (d) o produto dos elementos do vetor
   (e) a média dos elementos do vetor

Resposta:

```c
#include <stdio.h>
#include <stdlib.h>

int maior(int v[], int n){
    int x;
    if ( n==1) return v[0];
    x = maior(v,n-1);
    if (x > v[n-1]) return x;
    else return v[n-1];
}

int menor(int v[], int n){
    int x;
    if ( n==1) return v[0];
    x = menor(v,n-1);
    if (x < v[n-1]) return x;
    else return v[n-1];
}

int soma(int v[], int n){
    if ( n == 1) return v[0];
    return v[n-1] + soma(v,n-1);
}

int produto(int v[], int n){
    if ( n == 1) return v[0];
    return v[n-1] * produto(v,n-1);
}

float media(int v[], int n){
    float m;
    if (n == 1) return (v[0]);
    m = media (v,n-1);
    return ( (m * (n-1) + v[n-1])/(float) n);
}

int main(){

    int A[] = {1,2,3,4,5,6,7,8,9,10};

    printf("Maximo: %d\n",maior(A,10));
    printf("Minimo: %d\n",menor(A,10));
    printf("Soma: %d\n",soma(A,10));
    printf("Produto: %d\n",produto(A,10));
    printf("Media: %.2f\n",media(A,10));

    return 0;
}
```

5 Algoritmos de Ordenação

1. Faça uma descrição alto nível de cada um dos algoritmos estudados, levando em consideração a estabilidade e a ordem de complexidade (comparações e movimentações).

2. Dada a sequência de números: 3 4 9 2 5 8 2 1 7 4 6 2 9 8 5 1, ordene-a em ordem crescente segundo os seguintes algoritmos, apresentando a sequência obtida após cada passo do algoritmo:

(a) Bolha
(b) Seleção
(c) Inserção
(d) Mergesort
(e) QuickSort

3. João diz ter desenvolvido um algoritmo que é capaz de ordenar qualquer conjunto de n números reais, fazendo apenas $O(n^{\frac{3}{2}})$ comparações. Você compraria este algoritmo? Justifique.

   Resposta: Não. Pois existem algoritmos melhores já desenvolvidos na literatura, como o QuickSort e o MergeSort que tem complexidade $O(m \log n)$.

   Exemplo: $n = 100$, Usando MergeSort $O(64 * log_2(64)) = O(64 * 6) = O(384)$, usando o algoritmo do João $O(64^{3/2}) = O(512)$

4. Um amigo lhe diz que é capaz de ordenar qualquer conjunto de 6 números com no máximo 8 comparações. O seu amigo está falando a verdade ou mentindo? Justifique.

   Resposta:

5. Escreva uma versão recursiva dos algoritmos de ordenação Bolha, Seleção e Inserção.

```c
void bolha_recursivo(int *A, int n){

    bool trocou=false;

    for(int i=0;i<n-1;i++){
        if(A[i] > A[i+1]){
            int temp = A[i];
            A[i] = A[i+1];
            A[i+1] = temp;
            trocou=true;
        }
    }

    if(trocou) bolha_recursivo(A,n-1);
}

void selecao_recursivo(int *A, int inicio, int n){

    if(inicio < n){

        int min = inicio;

        for(int i=inicio+1; i<n; i++)
            if(A[i] < A[min])
                min = i;

        int temp = A[inicio];
        A[inicio] = A[min];
        A[min] = temp;

        selecao_recursivo(A, inicio+1, n);
    }

}

void inserca_recursivo(int *A, int n){

```

```
    if(n > 0){
        inserca_recursivo(A, n-1);
        int x = A[n];
        int i = n-1;
        while(i >=0 && A[i]>x){
            A[i+1] = A[i];
            i--;
        }
        A[i+1]=x;
    }
}
```